11. 병렬 처리 및 멀티스레딩 분석

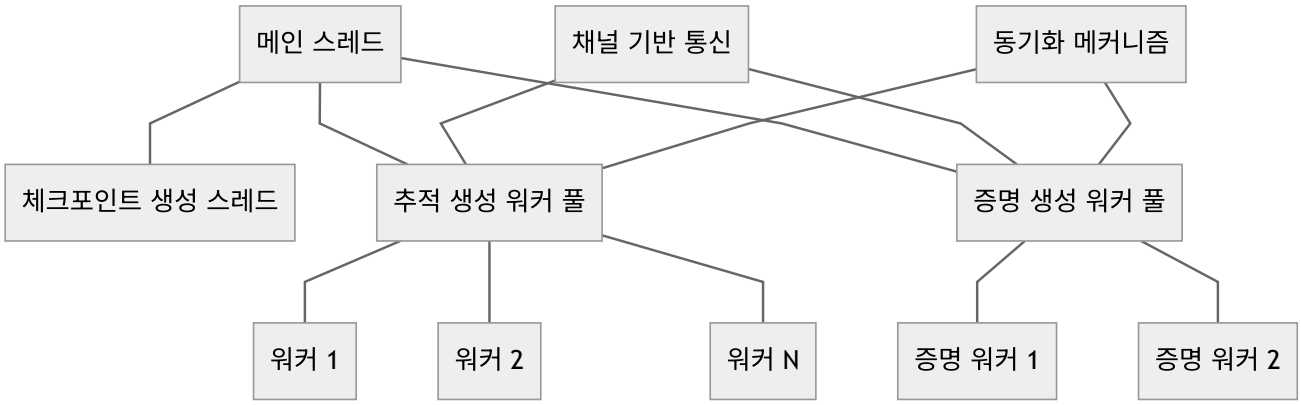
개요

SP1은 증명 생성 과정에서 다양한 병렬 처리 기법을 사용하여 성능을 최적화합니다. 이 문서는 SP1의 샤드 병렬 처리 구현 방식, 스레드 풀 관리 전략, 그리고 동기화 메커니즘을 상세히 분석합니다.

병렬 처리 아키텍처

전체 병렬 처리 구조

SP1의 병렬 처리는 다음과 같은 계층적 구조로 구성됩니다:



주요 병렬 처리 단계

1. Core Proof 생성 병렬화

```
// crates/prover/src/lib.rs

pub fn prove_core(&self, ...) -> Result<SP1CoreProof, SP1CoreProverError> {
```

```
// 두 개의 병렬 스레드 실행
std::thread::scope(|s| {
let (proof_tx, proof_rx) = channel();
let (shape_tx, shape_rx) = channel();


// 스레드 1: 실제 증명 생성
let  handle = s.spawn(move || {
sp1_core_machine::utils::prove_core_stream::<_, C::CoreProver>(
&self.core_prover,
pk,
program,
stdin,
opts.core_opts,
context,
self.core_shape_config.as_ref(),
proof_tx,
shape_tx,
None,
gas_calculator,
)
});


// 스레드 2: 재귀 프로그램 컴파일
```

```
for _ in 0..3 {
if let Ok((shape, is_complete)) = shape_rx.recv() {
let recursion_shape = SP1RecursionShape {
proof_shapes: vec![shape],
is_complete
};
let compress_shape = SP1CompressProgramShape::Recursion(recursion_shape);
self.program_from_shape(compress_shape, None);
}
}

// 결과 수집
let shard_proofs: Vec<ShardProof<_>> = proof_rx.iter().collect();
let (public_values_stream, cycles) = handle.join().unwrap().unwrap();
})
}
```

2. 압축 과정 병렬화

```
pub fn compress(&self, ...) -> Result<SP1ReduceProof<InnerSC>, SP1RecursionPro
let span = tracing::Span::current().clone();
thread::scope(|s| {
// 입력 전송 워커
let (input_tx, input_rx) = sync_channel::<(usize, usize, SP1CircuitWitness, bool
```

```
opts.recursion_opts.checkpoints_channel_capacity,
);
// 레코드/추적 생성 워커들
for _ in 0..opts.recursion_opts.trace_gen_workers {
let record_and_trace_sync = Arc::clone(&record_and_trace_sync);
let record_and_trace_tx = Arc::clone(&record_and_trace_tx);
let input_rx = Arc::clone(&input_rx);
s.spawn(move || {
// 추적 생성 작업 수행
loop {
let received = { input_rx.lock().unwrap().recv() };
if let Ok((index, height, input, false)) = received {
// 프로그램 실행 및 추적 생성
let record = execute_runtime(program, witness_stream);
let traces = self.compress_prover.generate_traces(&record);
// 결과 전송
record_and_trace_tx.lock().unwrap().send((
index, height,
TracesOrInput::ProgramRecordTraces(Box::new((
program, record, traces,
)))
)).unwrap();
}
```

```
}
});
}

// 증명 생성 워커들
for _ in 0..opts.recursion_opts.shard_batch_size {
s.spawn(move || {
// 증명 생성 작업 수행
});
}
})
}
```

워커 스레드 관리

동적 워커 수 조정

SP1은 시스템 리소스에 따라 워커 스레드 수를 동적으로 조정합니다:

```
// crates/stark/src/opts.rs
impl SP1ProverOpts {
pub fn cpu(cpu_ram_gb: usize) -> Self {
let mut opts = SP1ProverOpts::default();
// CPU 메모리에 따른 워커 수 조정
```

```
match cpu_ram_gb {
0..33 => {
opts.core_opts.trace_gen_workers = 1;
opts.core_opts.records_and_traces_channel_capacity = 1;
},
33..49 => {
opts.core_opts.trace_gen_workers = 2;
opts.core_opts.records_and_traces_channel_capacity = 2;
},
49..65 => {
opts.core_opts.trace_gen_workers = 4;
opts.core_opts.records_and_traces_channel_capacity = 4;
},
_ => {
opts.core_opts.trace_gen_workers = 8;
opts.core_opts.records_and_traces_channel_capacity = 8;
}
}
opts
}

pub fn gpu(cpu_ram_gb: usize, gpu_ram_gb: usize) -> Self {
let mut opts = SP1ProverOpts::default();
```

```
if 24 <= gpu_ram_gb {
opts.core_opts.trace_gen_workers = 4;
opts.core_opts.records_and_traces_channel_capacity = 4;
if cpu_ram_gb <= 20 {
opts.core_opts.trace_gen_workers = 2;
opts.core_opts.records_and_traces_channel_capacity = 1;
}
}
opts
}
}
```

환경 변수를 통한 세밀한 제어

```
pub struct SP1CoreOpts {
/// 추적 생성 워커 수
pub trace_gen_workers: usize,
/// 체크포인트 채널 용량
pub checkpoints_channel_capacity: usize,
/// 레코드/추적 채널 용량
pub records_and_traces_channel_capacity: usize,
}
```

```
impl Default for SP1CoreOpts {
fn  default() -> Self {
Self {
trace_gen_workers: env::var("TRACE_GEN_WORKERS").map_or_else(
|_| DEFAULT_TRACE_GEN_WORKERS,
|s| s.parse::<usize>().unwrap_or(DEFAULT_TRACE_GEN_WORKERS),
),
checkpoints_channel_capacity: env::var("CHECKPOINTS_CHANNEL_CAPACITY").map_or_el
|_| DEFAULT_CHECKPOINTS_CHANNEL_CAPACITY,
|s| s.parse::<usize>().unwrap_or(DEFAULT_CHECKPOINTS_CHANNEL_CAPACITY),
),
records_and_traces_channel_capacity: env::var("RECORDS_AND_TRACES_CHANNEL_CAPACI
.map_or_else(
|_| DEFAULT_RECORDS_AND_TRACES_CHANNEL_CAPACITY,
|s| s.parse::<usize>().unwrap_or(DEFAULT_RECORDS_AND_TRACES_CHANNEL_CAPACITY),
),
}
}
}
```

채널 기반 통신 시스템

동기 채널 (Sync Channel) 사용

SP1은 백프레셔(backpressure) 제어를 위해 동기 채널을 사용합니다:

```
// 체크포인트 채널
let (checkpoint_tx, checkpoint_rx) = sync_channel::<(usize, File, bool, u64)>(
opts.checkpoints_channel_capacity
);

// 레코드/추적 채널
let (record_and_trace_tx, record_and_trace_rx) = sync_channel::<(usize, usize, T
opts.records_and_traces_channel_capacity
);

// 증명 채널
let (proofs_tx, proofs_rx) = sync_channel::<(usize, usize, StarkVerifyingKey<Inn
num_first_layer_inputs * 2
);
```

채널 용량 최적화

```
const DEFAULT_TRACE_GEN_WORKERS: usize = 1;
const DEFAULT_CHECKPOINTS_CHANNEL_CAPACITY: usize = 128;
const DEFAULT_RECORDS_AND_TRACES_CHANNEL_CAPACITY: usize = 1;
```

```
// 메모리 사용량에 따른 채널 용량 조정
fn adjust_channel_capacity(cpu_ram_gb: usize) -> usize {
match cpu_ram_gb {
0..16 => 1, // 낮은 메모리: 작은 버퍼
16..32 => 2, // 중간 메모리: 중간 버퍼
32..64 => 4, // 높은 메모리: 큰 버퍼
_ => 8, // 매우 높은 메모리: 최대 버퍼
}
}
```

동기화 메커니즘

TurnBasedSync 구현

SP1은 순서 보장이 필요한 작업을 위해 턴 기반 동기화를 사용합니다:

```
// sp1_core_machine::utils::concurrency::TurnBasedSync 사용
let input_sync = Arc::new(TurnBasedSync::new());
let record_and_trace_sync = Arc::new(TurnBasedSync::new());
let proofs_sync = Arc::new(TurnBasedSync::new());

// 워커에서 순서 보장
s.spawn(move || {
loop {
```

```
let received = { input_rx.lock().unwrap().recv() };
if let Ok((index, height, input, false)) = received {
// 작업 수행
let result = process_input(input);
// 순서 대기
record_and_trace_sync.wait_for_turn(index);
// 결과 전송
record_and_trace_tx.lock().unwrap().send((
index, height, result
)).unwrap();
// 다음 턴으로 진행
record_and_trace_sync.advance_turn();
}
}
});
```

Arc<Mutex<>> 패턴

공유 리소스 접근을 위한 뮤텍스 사용:

```
// 채널 송신자/수신자 공유
let input_tx = Arc::new(Mutex::new(input_tx));
let input_rx = Arc::new(Mutex::new(input_rx));
let record_and_trace_tx = Arc::new(Mutex::new(record_and_trace_tx));
```

```
let record_and_trace_rx = Arc::new(Mutex::new(record_and_trace_rx));


// 워커 스레드에서 안전한 접근
for _ in 0..opts.recursion_opts.trace_gen_workers {
let input_rx = Arc::clone(&input_rx);
let record_and_trace_tx = Arc::clone(&record_and_trace_tx);
s.spawn(move || {
while let Ok(data) = input_rx.lock().unwrap().recv() {
let result = process_data(data);
record_and_trace_tx.lock().unwrap().send(result).unwrap();
}
});
}
```

Rayon을 통한 데이터 병렬성

p3-maybe-rayon 활용

SP1은 Plonky3의 `p3-maybe-rayon` 크레이트를 통해 조건부 병렬 처리를 지원합니다:

```
// Cargo.toml에서 rayon 의존성
[dependencies]
p3-maybe-rayon = "0.2.3-succinct"
rayon = "1.10.0"
```

```
rayon-scan = "0.1.1"


// 조건부 병렬 처리
#[cfg(feature = "parallel")]
use rayon::prelude::*;


// 벡터 병렬 처리
fn  parallel_process<T: Send + Sync>(data: &[T]) -> Vec<ProcessedData> {
#[cfg(feature = "parallel")]
{
data.par_iter().map(|item| process_item(item)).collect()
}
#[cfg(not(feature = "parallel"))]
{
data.iter().map(|item| process_item(item)).collect()
}
}
```

행렬 연산 병렬화

```
// 행렬 연산에서 Rayon 활용
use p3_maybe_rayon::prelude::*;
```

```rust
impl Matrix {
fn parallel_multiply(&self, other: &Matrix) -> Matrix {
let rows: Vec<_> = (0..self.height())
.into_par_iter() // 병렬 반복자 사용
.map(|i| {
(0..other.width())
.map(|j| self.row_dot_product(i, other.column(j)))
.collect()
})
.collect();
Matrix::from_rows(rows)
}
}
```

샤드 병렬 처리 전략

샤드 배치 처리

```rust
pub struct SP1CoreOpts {
/// 개별 샤드 크기
pub shard_size: usize,
/// 배치로 처리할 샤드 수
pub shard_batch_size: usize,
```

```
}

// 샤드 배치 병렬 처리
fn process_shard_batch(shards: Vec<Shard>, batch_size: usize) -> Vec<Proof> {
shards
.chunks(batch_size)
.collect::<Vec<_>>()
.into_par_iter() // 배치 단위로 병렬 처리
.flat_map(|batch| {
batch.into_par_iter() // 배치 내에서도 병렬 처리
.map(|shard| prove_shard(shard))
.collect::<Vec<_>>()
})
.collect()
}
```

동적 부하 분산

```
// 작업 큐 기반 동적 부하 분산
fn dynamic_load_balancing(tasks: Vec<Task>, num_workers: usize) -> Vec<Result>
let (task_tx, task_rx) = sync_channel(num_workers * 2);
let (result_tx, result_rx) = channel();
// 작업 분배 스레드
```

```
thread::spawn(move || {
for task in tasks {
task_tx.send(task).unwrap();
}
});
// 워커 스레드들
let task_rx = Arc::new(Mutex::new(task_rx));
let handles: Vec<_> = (0..num_workers)
.map(|_| {
let task_rx = Arc::clone(&task_rx);
let result_tx = result_tx.clone();
thread::spawn(move || {
while let Ok(task) = task_rx.lock().unwrap().recv() {
let result = process_task(task);
result_tx.send(result).unwrap();
}
})
})
.collect();
// 결과 수집
result_rx.iter().take(tasks.len()).collect()
}
```

성능 최적화 기법

작업 분할 전략

```
// CPU 집약적 작업의 세밀한 분할
fn fine_grained_parallelism<T: Send + Sync>(
data: &[T],
chunk_size: usize
) -> Vec<ProcessedData> {
data.par_chunks(chunk_size)
.flat_map(|chunk| {
chunk.par_iter()
.map(|item| expensive_computation(item))
.collect::<Vec<_>>()
})
.collect()
}

// 메모리 집약적 작업의 큰 단위 분할
fn coarse_grained_parallelism<T: Send + Sync>(
data: Vec<T>,
num_threads: usize
) -> Vec<ProcessedData> {
```

```
let chunk_size = data.len() / num_threads;
data.into_par_iter()
.chunks(chunk_size)
.map(|chunk| {
// 큰 단위로 순차 처리
chunk.into_iter()
.map(|item| memory_intensive_computation(item))
.collect::<Vec<_>>()
})
.flatten()
.collect()
}
```

캐시 친화적 처리

```
// 캐시 지역성을 고려한 데이터 배치
fn cache_friendly_processing(matrix: &Matrix) -> ProcessedMatrix {
let block_size = 64; // 캐시 라인 크기 고려
(0..matrix.height())
.step_by(block_size)
.into_par_iter()
.flat_map(|row_start| {
(0..matrix.width())
```

```
.step_by(block_size)
.into_par_iter()
.map(move |col_start| {
process_block(
matrix,
row_start,
col_start,
block_size
)
})
})
.collect()
}
```

오류 처리 및 복구

패닉 처리

```
// 워커 스레드 패닉 처리
fn robust_parallel_processing(tasks: Vec<Task>) -> Vec<Result<Output, Error>> {
let (panic_tx, panic_rx) = channel();
let results: Vec<_> = tasks
.into_par_iter()
```

```
.map(|task| {
let panic_tx = panic_tx.clone();
// 패닉 캐치
match catch_unwind(AssertUnwindSafe(|| process_task(task))) {
Ok(result) => result,
Err(panic_info) => {
panic_tx.send(format!("Task panicked: {:?}", panic_info)).ok();
Err(Error::TaskPanicked)
}
}
})
.collect();
// 패닉 로그 수집
for panic_msg in panic_rx.try_iter() {
tracing::error!("Worker panic: {}", panic_msg);
}
results
}
```

재시도 메커니즘

```
// 실패한 작업 재시도
fn retry_failed_tasks(tasks: Vec<Task>, max_retries: usize) -> Vec<Output> {
```

```
let mut remaining_tasks = tasks;

let mut results = Vec::new();

for attempt in 0..=max_retries {

let (success, failures): (Vec<_>, Vec<_>) = remaining_tasks

.into_par_iter()

.map(|task| process_task_with_result(task))

.partition(|result| result.is_ok());

results.extend(success.into_iter().map(|r| r.unwrap()));

if failures.is_empty() {

break;

}

if attempt < max_retries {

tracing::warn!("Retrying {} failed tasks (attempt {})", failures.len(), attempt

remaining_tasks = failures.into_iter().map(|r| r.unwrap_err()).collect();

} else {

tracing::error!("Failed to complete {} tasks after {} attempts", failures.len(),

}

}

results

}
```

성능 모니터링 및 튜닝

스레드 활용률 모니터링

```rust
use std::sync::atomic::{AtomicUsize, Ordering};

struct ThreadPoolMetrics {
active_threads: AtomicUsize,
completed_tasks: AtomicUsize,
total_processing_time: AtomicUsize,
}

impl ThreadPoolMetrics {
fn  record_task_start(&self) {
self.active_threads.fetch_add(1, Ordering::Relaxed);
}
fn  record_task_completion(&self, processing_time: Duration) {
self.active_threads.fetch_sub(1, Ordering::Relaxed);
self.completed_tasks.fetch_add(1, Ordering::Relaxed);
self.total_processing_time.fetch_add(
processing_time.as_millis() as usize,
Ordering::Relaxed
);
}
fn  utilization_rate(&self) -> f64 {
```

```
let active = self.active_threads.load(Ordering::Relaxed);
let total_threads = num_cpus::get();
active as f64 / total_threads as f64
}
}
```

동적 스레드 수 조정

```
struct AdaptiveThreadPool {
current_workers: AtomicUsize,
max_workers: usize,
min_workers: usize,
metrics: ThreadPoolMetrics,
}

impl AdaptiveThreadPool {
fn adjust_worker_count(&self) {
let utilization = self.metrics.utilization_rate();
let current = self.current_workers.load(Ordering::Relaxed);
let new_count = if utilization > 0.9 && current < self.max_workers {
current + 1 // 워커 수 증가
} else if utilization < 0.3 && current > self.min_workers {
current - 1 // 워커 수 감소
```

```
} else {
current  // 현재 상태 유지
};
if  new_count != current {
self.current_workers.store(new_count, Ordering::Relaxed);
tracing::info!("Adjusted worker count: {} -> {}", current, new_count);
}
}
}
```

환경 설정 및 튜닝 가이드

환경 변수 설정

```
# 워커 스레드 수 설정
export  TRACE_GEN_WORKERS=4

# 채널 용량 설정
export  CHECKPOINTS_CHANNEL_CAPACITY=256
export  RECORDS_AND_TRACES_CHANNEL_CAPACITY=8

# 샤드 배치 크기 설정
export  SHARD_BATCH_SIZE=4
```

```
# Rayon 스레드 풀 크기 설정
export RAYON_NUM_THREADS=8
```

시스템별 최적 설정

저사양 시스템 (4코어, 8GB RAM)

```
SP1ProverOpts {
core_opts: SP1CoreOpts {
trace_gen_workers: 2,
shard_batch_size: 1,
checkpoints_channel_capacity: 32,
records_and_traces_channel_capacity: 1,
},
recursion_opts: SP1CoreOpts {
trace_gen_workers: 1,
shard_batch_size: 1,
// ...
}
}
```

고사양 시스템 (16코어, 64GB RAM)

```
SP1ProverOpts {
```

```
core_opts: SP1CoreOpts {
trace_gen_workers: 8,
shard_batch_size: 4,
checkpoints_channel_capacity: 256,
records_and_traces_channel_capacity: 8,
},
recursion_opts: SP1CoreOpts {
trace_gen_workers: 4,
shard_batch_size: 2,
// ...
}
}
```

결론

SP1의 병렬 처리 및 멀티스레딩 시스템은 다음과 같은 특징을 가집니다:

1. **계층적 병렬화**: 작업 단계별로 적절한 병렬화 전략 적용
2. **동적 리소스 관리**: 시스템 리소스에 따른 자동 워커 수 조정
3. **채널 기반 통신**: 백프레셔 제어가 가능한 동기 채널 사용
4. **순서 보장**: TurnBasedSync를 통한 결정적 실행 순서 보장
5. **오류 복구**: 패닉 처리 및 재시도 메커니즘
6. **성능 모니터링**: 실시간 성능 지표 수집 및 조정

이러한 설계를 통해 SP1은 다양한 하드웨어 환경에서 최적의 성능을 달성하면서도 안정성과 확장성을 보장합니다.